



[Project Name] 開発スタートアップガイド

ようこそ！

このプロジェクトでは、「コードとドキュメントの一致 (Doc as Code)」を大切にしています。

開発スピードを落とさず、品質を保つため、以下のフローに従って開発を進めてください。

1. 準備するもの

- **GitHubアカウント**：組織への招待を受けてください。
- **ガイドライン**：詳細は「統合運用ガイドライン」を参照してください。

2. 開発のゴールデnrル 3選

1. コードを変えたら、その場でドキュメントも変更する、(Docs as Code)
2. ブランチ名の Prefix (`feature/` 等) は絶対に守る。(Naming)
3. バージョンタグは自動で作られるので、手動で作らない。(No Manual Tag)

3. 日々の開発フロー (チートシート)

Step1. タスク確認 & Issue化

- GitHub Project を開き、担当タスクを確認します。
- Issueの `Doc Status` フィールドを設定してください。
 - 仕様変更がある？ → `Required`

- リファクタリングのみ？ → `Not Needed`

Step2. ブランチ作成

Issueからブランチを作成するか、以下のルールでローカル作成します。

- 新機能 → `feature/機能名`
- バグ修正 → `bugfix/バグ名`
- ドキュメントのみ → `docs/内容`

Step3. 実装 (Code + Docs)

ここが最重要ポイントです。

- `src/` (ソースコード) を修正したら.....
- 同じコミット/同じPRで `docs/` (仕様書等) も修正してください。
- コミットメッセージ例： `feat(auth): ログイン機能を追加`

Step4. Pull Request (PR)

- PRテンプレートに従って、記入します。
- `Closes #123` のようにIssue番号を紐づけます。
- チェックポイント：
 - CI (ビルド・テスト) は通っているか？
 - プレビュー環境で、ドキュメントが崩れていないか？

Step5. レビュー&マージ

- レビューアー1名の承認をもらいます。(セルフマージ禁止)
- `Squash and Merge` ボタンを押してマージ。
- 自動でリリースされ、ポータルサイトが更新されます。

4. 困ったときは？

CI が落ちた

詳細ログを確認し、Lintエラーやテスト失敗を修正してください。

ドキュメント更新を忘れた

`Audit List` ビューで「実装完了・ドキュメント未完」として晒されてしまいます。
すぐに `docs/` 更新のPRを出してください。